

ASTROFÍSICA COMPUTACIONAL · OAN · 2026-I PROJECT ASSIGNMENT — SECTION 3

Astronomical Data Analysis: from Query to Dashboard

Dr. rer. nat. Jose Ivan Campos Rozo · jicamposr@unal.edu.co

1 Overview & Learning Goals

In this project you will build a complete, end-to-end data pipeline: query a real astronomical database, ingest the results into a local SQLite database, perform SQL-based statistical analysis, and produce a multi-panel dashboard inside a Jupyter Notebook. The pipeline mirrors the professional workflow of modern data-driven astrophysics research.

By the end of this project you will be able to:

- Query real astronomical catalogs programmatically using Astroquery or SunPy.
- Design and populate a normalized SQLite relational database with physical parameters.
- Retrieve, filter and aggregate data using SQL queries from Python (sqlite3 / pandas).
- Produce publication-quality statistical visualizations: histograms, box plots, pie charts, scatter plots with fits, correlation matrices.
- Interpret the statistical results in the context of the chosen astrophysical domain.

Deliverables

1. Jupyter Notebook (.ipynb) — full pipeline, all plots, written interpretations.
2. SQLite database file (.db) — generated and populated entirely by your notebook.
3. Short written report (PDF, 3–5 pages, opt) — introduction, methods, results, conclusions.
4. requirements.txt or environment.yml — ensures the notebook is fully reproducible.

2 Dataset Options (choose one)

Select one of the four datasets below. All options follow the same five-stage pipeline and are evaluated with the same rubric. Choose the dataset most aligned with your research interests or thesis topic.

Option A — Solar Active Regions (HEK / SunPy)

Tool	SunPy · Fido interface + Heliophysics Event Knowledgebase (HEK)
Parameters	NOAA AR number, heliographic coordinates (Stonyhurst), area at disk center [Mm^2], magnetic classification ($\alpha/\beta/\gamma/\delta$), associated flare count, peak GOES X-ray flux [W/m^2]
Required plots	Pie (magnetic class distribution), scatter (area vs flare count + linear fit), histogram (peak flux, log scale + power law), box plots (area by magnetic class), time series of AR count per Carrington rotation
Difficulty	Intermediate, 2 weeks, Solar physics emphasis

Option B — Stellar Populations in Open Clusters (Gaia DR3)

Tool	Astroquery · Gaia TAP+ (cone search around a known open cluster, e.g. Pleiades or Hyades)
Parameters	Parallax [mas], proper motion ($\mu_{\alpha*}, \mu_{\delta}$) [mas/yr], G magnitude, BP–RP color [mag], Teff [K], stellar radius [$R_{\odot}$], luminosity [$L_{\odot}$]
Required plots	H-R diagram (scatter, color-coded by Teff), parallax histogram + Gaussian fit, proper motion vector diagram, luminosity function (histogram + power-law fit), correlation heatmap of physical parameters
Difficulty	Intermediate, 2 weeks, Stellar astrophysics emphasis

Option C — Galaxy Scaling Relations (SDSS DR17)

Tool	Astroquery · SDSS SQL (CasJobs-style query via astroquery.sdss), volume-limited galaxy sample
Parameters	Redshift z , stellar mass $\log(M_{\star}/M_{\odot})$, effective radius R_e [kpc], Sérsic index n , $g-r$ color [mag], star formation rate SFR [$M_{\odot}/\text{yr}$], morphological type (early/late)
Required plots	Morphology distribution (pie + bar), mass–size relation (scatter + power law), color–magnitude diagram, SFR histogram by morphology (box), red sequence vs blue cloud separation, Pearson correlation heatmap
Difficulty	Advanced, 2–3 weeks, Extragalactic astrophysics emphasis

Option D — Exoplanet Demographics (NASA Exoplanet Archive)

Tool	Astroquery · NASA Exoplanet Archive TAP (pscomppars table, confirmed planets only)
Parameters	Orbital period P [days], semi-major axis a [AU], planet radius R_p [$R_\oplus$], mass M_p [$M_\oplus$], equilibrium temperature T_{eq} [K], host star spectral type, detection method, discovery year
Required plots	Detection method distribution (pie + bar), period–radius diagram (scatter), mass–radius relation (scatter + fit), equilibrium temperature histogram by zone (cold/temperate/hot), box plots of R_p by host star type, discovery timeline (cumulative count by year and method)
Difficulty	Intermediate, 2 weeks, Exoplanetary science emphasis

3 Full Pipeline — Pseudocode & Scaffolding

The pipeline is structured in five sequential stages. Each stage corresponds to a clearly labelled section of your Jupyter Notebook. The pseudocode below is intentionally dataset-agnostic. Concrete, runnable code for each option is provided in Section 4.

3.1 Stage 1 — Data Acquisition

```
# STAGE 1 --- QUERY THE ASTRONOMICAL DATABASE
IMPORT astroquery / sunpy / requests
IMPORT astropy.units as u
IMPORT astropy.coordinates.SkyCoord

DEFINE target      <- object name / sky region / time window
DEFINE columns     <- list of physical parameters to retrieve
DEFINE quality_flags <- detection confidence, error thresholds

raw_table <- SERVICE.query(target, columns, quality_flags)

IF len(raw_table) == 0:
    RAISE QueryError('No objects returned --- check parameters')
CHECK expected_columns subset of raw_table.colnames
REMOVE rows WHERE critical_column IS NULL OR error_flag IS TRUE

raw_table.write('raw_data.csv', overwrite=True)
PRINT f'Retrieved {len(raw_table)} objects after quality cuts'
```

Notebook Markdown prompts:

- How many objects did the query return before and after quality cuts?
- Are there known selection biases in this catalog (flux limit, sky coverage)?
- What time window / sky region does the sample cover?

3.2 Stage 2 — Database Design & Ingestion

```

# STAGE 2 --- CREATE AND POPULATE THE SQLite DATABASE
IMPORT sqlite3
IMPORT pandas as pd
IMPORT datetime

conn <- sqlite3.connect('astro_project.db')
cur <- conn.cursor()

cur.execute('''
CREATE TABLE IF NOT EXISTS objects (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    param_1 REAL,
    param_2 REAL,
    param_3 REAL,
    category TEXT,
    retrieved_at TEXT
)
''')

cur.execute('''
CREATE TABLE IF NOT EXISTS metadata (
    key TEXT PRIMARY KEY,
    value TEXT
)
''')

FOR row IN raw_table:
    cur.execute(
        'INSERT INTO objects (name, param_1, param_2, param_3, category,
        retrieved_at) VALUES (?, ?, ?, ?, ?, ?)',
        (row.name, row.p1, row.p2, row.p3, row.cat, datetime.datetime.
        utcnow().isoformat())
    )

conn.commit()
conn.close()

```

Design tips:

- Always use REAL for physical quantities; never embed units in the number.
- Use CREATE TABLE IF NOT EXISTS so re-running does not fail.
- The metadata table makes your DB self-documenting and traceable.

3.3 Stage 3 — SQL Queries & Data Retrieval

```

# STAGE 3 --- QUERY THE DATABASE AND LOAD INTO PANDAS
conn <- sqlite3.connect('astro_project.db')

df <- pd.read_sql('SELECT * FROM objects', conn)

```

```

df_sub <- pd.read_sql(
    '''SELECT name, param_1, param_2, category
       FROM objects
       WHERE param_1 > ? AND category != ?''',
    conn, params=(threshold_value, 'exclude_class')
)

df_stats <- pd.read_sql(
    '''SELECT category,
           COUNT(*)      AS n,
           AVG(param_1)   AS mean_p1,
           MIN(param_1)   AS min_p1,
           MAX(param_1)   AS max_p1,
           AVG(param_2)   AS mean_p2
       FROM objects
       GROUP BY category
       ORDER BY n DESC''',
    conn
)

conn.close()
PRINT df_stats
PRINT df.describe()

```

SQL concepts exercised: SELECT, WHERE, GROUP BY, ORDER BY, COUNT, AVG, MIN, MAX, parameterized queries.

3.4 Stage 4 — Statistical Visualization Dashboard

```

# STAGE 4 --- MULTI-PANEL STATISTICAL DASHBOARD
IMPORT matplotlib.pyplot as plt
IMPORT seaborn as sns
IMPORT scipy.stats as stats
IMPORT numpy as np

fig, axes <- plt.subplots(3, 2, figsize=(14, 16))
fig.suptitle('Dashboard: [Your Dataset Name]', fontsize=16, fontweight='
    bold')

# Plot examples:
# 1) Category distribution (pie)
# 2) Histogram + Gaussian fit
# 3) Box plot by category
# 4) Scatter + linear regression
# 5) Empirical CDF by category
# 6) Pearson correlation heatmap

plt.tight_layout()
plt.savefig('dashboard.pdf', dpi=150, bbox_inches='tight')
plt.show()

```

Each plot must have axis labels with units, a descriptive title, and a Markdown cell below it with a physical interpretation (1–3 sentences minimum).

3.5 Stage 5 — Interpretation & Written Report

```
# STAGE 5 --- SCIENTIFIC INTERPRETATION
FOR each plot:
    WRITE Markdown cell:
        - What does the distribution or trend show physically?
        - Is it consistent with literature / theory?
        - What are the limitations?

WRITE summary section (3--5 findings + follow-up analysis + references)
WRITE standalone report.pdf (3--5 pages)
```

4 Concrete Code Examples per Option

Copy and adapt the relevant snippet as the starting point for Stage 1 of your notebook.

4.0.1 Option A — Solar Active Regions (SunPy + HEK)

```
FROM sunpy.net IMPORT Fido, attrs as a
IMPORT pandas as pd

result = Fido.search(
    a.Time('2014-01-01', '2014-12-31'),
    a.hek.EventType('AR'),
)

table = Fido.fetch(result)
df = table.to_pandas()
```

4.0.2 Option B — Stellar Populations (Gaia DR3)

```
FROM astroquery.gaia IMPORT Gaia
IMPORT astropy.units as u

job = Gaia.launch_job_async('''
SELECT source_id, ra, dec,
       parallax, parallax_over_error,
       pmra, pmdec,
       phot_g_mean_mag, bp_rp,
       teff_gspphot, radius_gspphot, lum_flame
FROM gaiadr3.gaia_source
WHERE CONTAINS(POINT(ra, dec), CIRCLE(56.75, 24.12, 2.0)) = 1
   AND parallax > 6.0
   AND parallax_over_error > 10
''')

table = job.get_results()
df = table.to_pandas()
```

4.0.3 Option C — Galaxy Scaling Relations (SDSS DR17)

```
FROM astroquery.sdss IMPORT SDSS

query = '''
SELECT p.objID, p.ra, p.dec,
       p.modelMag_g - p.modelMag_r AS color_gr,
       p.petroRad_r AS petro_radius_arcsec,
       p.fracDeV_r AS sersic_proxy,
       p.type AS phot_type,
       s.z AS redshift
FROM PhotoObj AS p
JOIN SpecObj AS s ON p.objID = s.bestObjID
WHERE s.z BETWEEN 0.02 AND 0.10
      AND s.zWarning = 0
      AND p.modelMag_r < 17.77
      AND p.type = 3
'''

table = SDSS.query_sql(query, timeout=120)
df = table.to_pandas()
```

4.0.4 Option D — Exoplanet Demographics (NASA Exoplanet Archive)

```
FROM astroquery.ipac.nexsci.nasa_exoplanet_archive IMPORT
NasaExoplanetArchive AS NEA
IMPORT numpy as np

table = NEA.query_criteria(
    table='pscomppars',
    select='pl_name, pl_orbper, pl_orbsmax, pl_rade, pl_bmasse, pl_eqt,
st_spectype, discoverymethod, disc_year',
    where='''pl_controv_flag = 0 AND pl_rade IS NOT NULL AND pl_bmasse
IS NOT NULL AND pl_eqt IS NOT NULL'''
)
df = table.to_pandas()

conditions = [df['pl_eqt'] < 273, df['pl_eqt'] < 373]
choices = ['Cold', 'Temperate']
df['temp_zone'] = np.select(conditions, choices, default='Hot')
```

5 Evaluation Rubric

Each of the six criteria is scored from 0 to 3. Maximum total: 18 points, converted to a 0–5.0 grade scale.

Criterion	0 — Insufficient	1 — Basic	2 — Proficient	3 — Excellent
1. Data acquisition & DB schema	Missing or broken pipeline	Queries schema incomplete	work; incom- Correct schema + metadata table	Schema + error handling + docs

2. SQL queries & Pandas integration	No SQL used	Basic only	SELECT	Filters, aggregations, GROUP BY	Complex queries, indices, efficient I/O
3. Statistical visualizations (≥ 5 plots)	0–2 plots, no labels	3–4 plots, partial labels		5 labeled plots with units	5+ plots, annotations, fits, color-bars
4. Scientific interpretation	No discussion	Superficial description only		Correct physical interpretation	Connects to literature + uncertainties
5. Code quality & reproducibility	Notebook does not run	Runs with errors / no comments		Runs cleanly, well commented	Modular, docstrings, requirements.txt
6. Report & written communication	Missing	Bullet points only		Structured report with captions	Publication-quality captions + refs

Points (/18)	Percentage	Grade (/5.0)
17–18	94–100%	4.5–5.0
14–16	78–89%	4.0–4.4
11–13	61–72%	3.5–3.9
8–10	44–56%	3.0–3.4
0–7	< 44%	< 3.0 (Fail)

6 Suggested Timeline (2 weeks)

Period	Tasks	Checkpoint
Days 1–2	Choose dataset, read API docs, execute first test query	Test query returning ≥ 50 clean objects
Days 3–4	Finalize query + cleaning, design DB schema, populate .db	Working astro_project.db with meta-data table
Days 5–7	SQL queries in Python, exploratory statistics, first 2–3 plots	Notebook Sections 1–3 complete
Days 8–10	Complete all 5+ plots, add fits/annotations/color coding	Full dashboard panel, saved as dashboard.pdf
Days 11–12	Write per-plot interpretations, draft report sections	Report draft (PDF) ready for self-review
Days 13–14	Polish notebook, peer check, add requirements.txt	Final zip package submitted to platform

7 Submission Instructions

- Required contents: `notebook.ipynb`, `astro_project.db`, `report.pdf`(opt), `requirements.txt`
- The notebook must execute top-to-bottom without errors in a fresh kernel.
- Upload to github before the announced deadline.

Academic Integrity

All code, analysis, and scientific interpretation must reflect your own work. External libraries and prior publications must be cited appropriately. Use of AI coding assistants is permitted as a tool; all scientific reasoning and interpretation must be entirely your own and demonstrable in a live oral defense.

Good luck — clear skies and clean queries!